

Strumenti

Algoritmi di Base

Introduzione agli algoritmi di ricerca e ordinamento più comuni, implementati in C++.

Cos'è un algoritmo?

Un **algoritmo** è una sequenza di passi ben definiti per risolvere un problema — come una ricetta di cucina.

Capire gli algoritmi di base ti insegna a pensare in modo strutturato e a valutare quanto "veloce" o "lento" è il tuo codice. Sono fondamentali in informatica.

Vediamo i quattro algoritmi più importanti per chi inizia: ricerca lineare, ricerca binaria, bubble sort e selection sort.

Ricerca Lineare

Come funziona

La **ricerca lineare** è la più semplice: guardi gli elementi uno per uno dall'inizio alla fine, finché non trovi quello che cerchi.

È come cercare il tuo nome in un elenco scritto a mano: leggi il primo nome, poi il secondo, poi il terzo... finché lo trovi o arrivi in fondo.

Implementazione

```
#include <iostream>
using namespace std;

// Cerca target nell'array. Restituisce la posizione (indice) oppure -1 se
// non trovato.
int ricercaLineare(int arr[], int dimensione, int target) {
    for (int i = 0; i < dimensione; i++) {
        if (arr[i] == target) {
            return i;    // trovato! restituisce la posizione
        }
    }
    return -1;          // non trovato
}

int main() {
    int numeri[] = {15, 3, 8, 42, 7, 19, 11};
    int dim = 7;
    int cercato = 42;

    int risultato = ricercaLineare(numeri, dim, cercato);

    if (risultato != -1) {
        cout << "Trovato " << cercato << " alla posizione " << risultato <<
endl;
    } else {
        cout << cercato << " non trovato" << endl;
    }

    return 0;
}
```

Output:

Trovato 42 alla posizione 3

Caratteristiche

- Funziona su array **non ordinati** — l'ordine non importa
- Nel caso peggiore (elemento non presente o all'ultimo posto): controlla tutti gli **n** elementi

- Semplicissimo da scrivere e capire

Ricerca Binaria

Come funziona

La **ricerca binaria** è molto più veloce, ma richiede che l'array sia **già ordinato**.

Pensa a quando cerchi una parola nel dizionario. Non inizi dalla prima pagina — apri il dizionario a metà. Se la parola viene prima, guardi nella prima metà; se viene dopo, guardi nella seconda metà. Ripeti, dimezzando ogni volta lo spazio di ricerca.

Passi:

1. Guarda l'elemento al centro dell'array
2. Se è quello cercato, hai finito
3. Se quello cercato è più piccolo, continua solo nella metà sinistra
4. Se quello cercato è più grande, continua solo nella metà destra
5. Ripeti finché trovi l'elemento o lo spazio si esaurisce

Implementazione

```
#include <iostream>
using namespace std;

int ricercaBinaria(int arr[], int dimensione, int target) {
    int sinistra = 0;
    int destra = dimensione - 1;

    while (sinistra <= destra) {
        int centro = sinistra + (destra - sinistra) / 2;

        if (arr[centro] == target) {
            return centro;          // trovato!
        } else if (arr[centro] < target) {
            sinistra = centro + 1;   // cerca nella metà destra
        } else {
            destra = centro - 1;     // cerca nella metà sinistra
        }
    }

    return -1; // non trovato
}

int main() {
    // IMPORTANTE: l'array deve essere ordinato!
    int numeri[] = {2, 5, 8, 12, 16, 23, 38, 45, 72, 91};
    int dim = 10;
    int cercato = 23;

    int risultato = ricercaBinaria(numeri, dim, cercato);

    if (risultato != -1) {
        cout << "Trovato " << cercato << " alla posizione " << risultato <<
endl;
    } else {
        cout << cercato << " non trovato" << endl;
    }

    return 0;
}
```

Output:

Trovato 23 alla posizione 5

Visualizzazione passo per passo

Cerchiamo 23 in {2, 5, 8, 12, 16, 23, 38, 45, 72, 91} :

Passo 1: centro = 4 $\rightarrow$ arr[4] = 16 < 23 $\rightarrow$ vai a destra (sinistra = 5)

Passo 2: centro = 7 $\rightarrow$ arr[7] = 45 > 23 $\rightarrow$ vai a sinistra (destra = 6)

Passo 3: centro = 5 $\rightarrow$ arr[5] = 23 $\rightarrow$ TROVATO!

Soli 3 passi su un array di 10 elementi. Con un milione di elementi, ne basterebbero circa 20.

Bubble Sort

Come funziona

Il **bubble sort** (ordinamento a bolla) scorre l'array più volte. A ogni passata, confronta gli elementi vicini a coppie e li scambia se sono nell'ordine sbagliato. I valori grandi "salgono" verso la fine, come bolle d'acqua.

Implementazione

```
#include <iostream>
using namespace std;

void bubbleSort(int arr[], int dimensione) {
    for (int i = 0; i < dimensione - 1; i++) {
        // Ogni passata sposta il valore più grande in fondo
        for (int j = 0; j < dimensione - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                // Scambia i due elementi vicini
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

void stampaArray(int arr[], int dimensione) {
    for (int i = 0; i < dimensione; i++) {
        cout << arr[i];
        if (i < dimensione - 1) cout << ", ";
    }
    cout << endl;
}

int main() {
    int numeri[] = {64, 34, 25, 12, 22, 11, 90};
    int dim = 7;

    cout << "Prima dell'ordinamento: ";
    stampaArray(numeri, dim);

    bubbleSort(numeri, dim);

    cout << "Dopo l'ordinamento: ";
    stampaArray(numeri, dim);

    return 0;
}
```

Output:

Prima dell'ordinamento: 64, 34, 25, 12, 22, 11, 90
Dopo l'ordinamento: 11, 12, 22, 25, 34, 64, 90

Visualizzazione di una passata

Array: {64, 34, 25, 12}

64 e 34: $64 > 34 \rightarrow$ scambia $\rightarrow$ {34, 64, 25, 12}
64 e 25: $64 > 25 \rightarrow$ scambia $\rightarrow$ {34, 25, 64, 12}
64 e 12: $64 > 12 \rightarrow$ scambia $\rightarrow$ {34, 25, 12, 64}

Il 64 è finito in fondo. Si ripete per il resto.

Selection Sort

Come funziona

Il **selection sort** (ordinamento per selezione) funziona in modo diverso:

1. Trova il valore più piccolo e mettilo in prima posizione
2. Trova il secondo valore più piccolo e mettilo in seconda posizione
3. Ripeti fino alla fine

Come ordinare le carte: ogni volta "scegli" la carta più bassa e la metti al suo posto.

Implementazione

```
#include <iostream>
using namespace std;

void selectionSort(int arr[], int dimensione) {
    for (int i = 0; i < dimensione - 1; i++) {
        // Trova il minimo nella parte non ancora ordinata
        int indiceMinimo = i;
        for (int j = i + 1; j < dimensione; j++) {
            if (arr[j] < arr[indiceMinimo]) {
                indiceMinimo = j;
            }
        }

        // Scambia il minimo trovato con l'elemento in posizione i
        if (indiceMinimo != i) {
            int temp = arr[i];
            arr[i] = arr[indiceMinimo];
            arr[indiceMinimo] = temp;
        }
    }
}

int main() {
    int numeri[] = {64, 25, 12, 22, 11};
    int dim = 5;

    cout << "Prima: ";
    for (int i = 0; i < dim; i++) cout << numeri[i] << " ";
    cout << endl;

    selectionSort(numeri, dim);

    cout << "Dopo: ";
    for (int i = 0; i < dim; i++) cout << numeri[i] << " ";
    cout << endl;

    return 0;
}
```

Output:

Prima: 64 25 12 22 11
Dopo: 11 12 22 25 64

Confronto tra gli algoritmi

Algoritmo	Array ordinato richiesto?	Velocità (caso peggiore)	Difficoltà
Ricerca Lineare	No	N confronti	Facilissima
Ricerca Binaria	Sì	~20 confronti per 1 milione di elementi	Moderata
Bubble Sort	No (ordina lui)	N ² confronti	Facile
Selection Sort	No (ordina lui)	N ² confronti	Facile

- Usa la **ricerca lineare** su array piccoli o non ordinati
- Usa la **ricerca binaria** su array grandi e già ordinati
- **Bubble sort** e **selection sort** sono utili per capire i concetti, ma lenti su array grandi

La soluzione pronta in C++: `std::sort`

Nella pratica, non scrivere mai bubble sort o selection sort in un programma vero. La libreria standard ha `sort()` che è molto più veloce:

```

#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;

int main() {
    vector<int> numeri = {64, 25, 12, 22, 11, 90, 38};

    sort(numeri.begin(), numeri.end()); // ordina in modo efficiente

    cout << "Ordinato: ";
    for (int n : numeri) cout << n << " ";
    cout << endl;

    // Ricerca binaria già pronta
    if (binary_search(numeri.begin(), numeri.end(), 22)) {
        cout << "22 trovato!" << endl;
    }

    return 0;
}

```

Output:

```

Ordinato: 11 12 22 25 38 64 90
22 trovato!

```

Studiare gli algoritmi "di base" ti insegna a ragionare passo per passo e a capire il concetto di efficienza — due delle basi più importanti dell'informatica.

Buone Pratiche di Codice

Come scrivere codice C++ leggibile, manutenibile e professionale.

Scrivere codice che si capisce

Scrivere codice che funziona è il primo traguardo. Scrivere codice che **si capisce, si modifica facilmente e non crea problemi nel tempo** è il secondo — ed è quello che distingue un programmatore che cresce da uno che rimane bloccato.

Queste abitudini ti aiutano a:

- Trovare i bug più in fretta
- Capire il tuo stesso codice dopo settimane senza toccarlo
- Lavorare con altri programmatori senza causare confusione
- Evitare gli errori classici che fanno perdere tempo

Usa nomi che spiegano da soli

Il nome di una variabile o di una funzione deve dire **cosa rappresenta** o **cosa fa**:

```
// Scadente: cosa sono x, y, b?  
int x, y;  
double d;  
bool b;  
  
// Buono: si capisce subito  
int larghezza, altezza;  
double prezzoUnitario;  
bool maggiorenne;
```

Lo stesso vale per le funzioni:

```
// Scadente  
void f(int a, int b) { return a + b; }  
  
// Buono: si capisce cosa fa senza leggere il corpo  
int calcolaSomma(int primo, int secondo) { return primo + secondo; }
```

Commenta il "perché", non il "cosa"

Il codice mostra già **cosa** fa. I commenti devono spiegare **perché** lo fa:

```
// Scadente: ridondante, il codice lo dice già
tentativi = tentativi + 1; // incrementa tentativi

// Buono: spiega il ragionamento
tentativi = tentativi + 1; // ogni risposta sbagliata aggiunge un tentativo
```

Documenta le funzioni complesse:

```
// Calcola il massimo comun divisore di due interi positivi.
// Precondizione: a > 0 e b > 0
// Usa l'algoritmo di Euclide (sottrazione ripetuta del resto).
int mcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}
```

Ogni funzione fa una sola cosa

Se una funzione è troppo lunga o fa troppe cose, spezzala. Una funzione ben fatta si capisce in pochi secondi:

```

// Scadente: una funzione enorme che legge, calcola e stampa tutto insieme
void elaboraStudenti() {
    // 100 righe con tutto mescolato...
}

// Buono: ogni funzione ha una responsabilità precisa
void leggiVoti(vector<int>& voti);
double calcolaMedia(const vector<int>& voti);
int trovaMassimo(const vector<int>& voti);
void stampaReport(const string& nome, double media, int max);

void elaboraStudenti() {
    vector<int> voti;
    leggiVoti(voti);
    double media = calcolaMedia(voti);
    int max = trovaMassimo(voti);
    stampaReport("Alice", media, max);
}

```

Evita i "numeri magici"

Un numero scritto direttamente nel codice senza spiegazione si chiama "numero magico". Chi lo legge non capisce cosa rappresenta:

```

// Scadente: cosa sono 86400 e 3.14159?
if (secondi > 86400) {
    area = 3.14159 * r * r;
}

// Buono: i nomi spiegano tutto
const int SECONDI_PER_GIORNO = 86400;
const double PI = 3.14159265358979;

if (secondi > SECONDI_PER_GIORNO) {
    area = PI * r * r;
}

```

Inizializza sempre le variabili

In C++, una variabile non inizializzata contiene un valore casuale (qualunque cosa ci fosse prima in quella zona di memoria):

```
// Pericoloso: valore casuale!
int x;
cout << x;    // stampa qualcosa di imprevedibile

// Sicuro
int x = 0;
cout << x;    // stampa 0
```

Valida l'input dell'utente

Non fidarti mai di quello che l'utente inserisce:

```
int eta;
cout << "Inserisci eta: ";
cin >> eta;

// Scadente: accetta qualsiasi valore, anche -500 o 9999
cout << "Hai " << eta << " anni";

// Buono: controlla prima di usare
if (eta < 0 || eta > 150) {
    cout << "Eta non valida!" << endl;
    return 1;
}
cout << "Hai " << eta << " anni";
```

Evita le variabili globali

Le variabili globali rendono il codice difficile da capire: chiunque, in qualunque punto del programma, può modificarle. Meglio passare i dati come parametri:

```
// Scadente: la variabile globale può essere modificata ovunque
int contatore = 0;
void incrementa() { contatore++; }

// Meglio: i dati vengono passati esplicitamente
void incrementa(int& contatore) { contatore++; }
```

Usa **const** per i dati che non cambiano

const comunica le intenzioni e previene modifiche accidentali:

```
// Parametri che la funzione non deve modificare
void stampa(const string& nome) {
    cout << nome << endl;
    // nome = "altro"; // il compilatore ti ferma se provi
}

// Valori fissi
const int MAX_STUDENTI = 30;
const double PI = 3.14159;
```

Verifica sempre le operazioni che possono fallire

Non assumere che un'operazione sia andata a buon fine:

```
// Scadente: apre il file senza verificare
ifstream file("dati.txt");
string riga;
getline(file, riga); // e se il file non esiste?

// Buono: controlla prima di usare
ifstream file("dati.txt");
if (!file.is_open()) {
    cerr << "Errore: impossibile aprire 'dati.txt'" << endl;
    return 1;
}
string riga;
getline(file, riga); // ora è sicuro
```

Non ripetere il codice (DRY)

DRY significa "Don't Repeat Yourself" — non ripeterti. Se scrivi lo stesso codice in più punti, estrailo in una funzione:

```
// Scadente: stessa struttura ripetuta
cout << "=====" << endl;
cout << "Alice: 8.5" << endl;
cout << "=====" << endl;
cout << "=====" << endl;
cout << "Bob: 7.2" << endl;
cout << "=====" << endl;

// Buono: una funzione riutilizzabile
void stampaScheda(const string& nome, double voto) {
    cout << "=====" << endl;
    cout << nome << ": " << voto << endl;
    cout << "=====" << endl;
}

stampaScheda("Alice", 8.5);
stampaScheda("Bob", 7.2);
```

Testa con casi diversi

Quando scrivi una funzione, testala con:

- Valori normali (caso standard)
- Valori limite (il minimo e il massimo consentiti)
- Valori non validi (cosa succede se riceve dati sbagliati?)

```
// Testa la funzione massimo con vari casi
cout << massimo(5, 3) << endl;    // 5 – caso normale
cout << massimo(-1, -5) << endl;  // -1 – numeri negativi
cout << massimo(7, 7) << endl;    // 7 – valori uguali
```

Debugging di Base

Tecniche e strumenti per trovare e correggere gli errori nei programmi C++.

Cosa significa fare debugging?

Tutti i programmatori scrivono codice con bug. Non è una questione di bravura — è normale. La differenza tra un programmatore esperto e uno alle prime armi spesso sta nella **velocità con cui trovano e correggono** gli errori.

Il debugging (eliminazione dei bug) è un'abilità che si impara con la pratica. Queste tecniche ti aiuteranno a trovare i problemi più velocemente.

I tre tipi di errori

- **Errori di compilazione:** il compilatore si rifiuta di creare il programma. Ti dice esattamente dove c'è il problema.
- **Errori di runtime:** il programma parte ma poi si blocca o crasha. Devi capire perché.
- **Errori logici:** il programma gira, non crasha, ma produce risultati sbagliati. I più subdoli.

Leggere i messaggi di errore del compilatore

Quando il compilatore trova un errore, ti dice esattamente dove:

```
hello.cpp:5:5: error: 'cout' was not declared in this scope
  cout << "Ciao!" << endl;
  ^~~~
```

Il formato è: `file:riga:colonna: tipo: messaggio`

- `hello.cpp` — il file con l'errore
- `5` — la riga dell'errore
- `error` — tipo di problema (`error` = grave, `warning` = avviso)
- Il messaggio spiega cosa c'è di sbagliato

Consiglio importante: correggi sempre il **primo** errore nella lista. Spesso un errore ne genera altri a cascata. Una volta sistemato il primo, molti degli altri spariscono.

I warning: segnali di pericolo

I warning non impediscono la compilazione, ma segnalano codice potenzialmente problematico:

```
warning: unused variable 'x' [-Wunused-variable]
```

Non ignorarli — spesso indicano errori logici nascosti.

Compila sempre con `-Wall` per vedere tutti i warning:

```
g++ programma.cpp -o programma -Wall
```

La tecnica più semplice: stampe di debug

Il modo più diretto per capire cosa fa il programma è aggiungere `cout` per stampare i valori delle variabili nei punti chiave:

```
double calcola(int a, int b) {  
    cout << "[DEBUG] a=" << a << ", b=" << b << endl; // stampa i valori  
    in entrata  
  
    double risultato = a / b; // possibile divisione intera!  
  
    cout << "[DEBUG] risultato=" << risultato << endl;  
  
    return risultato;  
}
```

Aggiungi il prefisso `[DEBUG]` così quando hai finito sai quali `cout` eliminare.

Gli errori più comuni

Variabile non inizializzata

```
int x;  
cout << x;    // stampa un numero casuale – bug!  
  
// Soluzione: inizializza sempre  
int x = 0;
```

Divisione intera indesiderata

```
int a = 7, b = 2;  
double risultato = a / b;    // 3.0, non 3.5! (divisione intera)  
  
// Soluzione: converti uno dei due prima di dividere  
double risultato = (double)a / b;    // 3.5
```

Errore di indice (off-by-one)

```
int arr[5] = {1, 2, 3, 4, 5};  
for (int i = 0; i <= 5; i++) {    // ERRORE: i arriva a 5, ma arr[5] non  
    cout << arr[i];  
}  
  
// Soluzione: usa < invece di <=  
for (int i = 0; i < 5; i++) {  
    cout << arr[i];  
}
```

= invece di **==** nelle condizioni

```
int x = 5;
if (x = 10) {    // ERRORE: assegna 10 a x, non confronta!
    cout << "uguale";
}

// Soluzione: usa == per confrontare
if (x == 10) {
    cout << "uguale";
}
```

Ciclo infinito

```
int i = 0;
while (i < 10) {
    cout << i;
    // manca i++! il ciclo non finisce mai
}

// Soluzione: aggiorna sempre la variabile del ciclo
while (i < 10) {
    cout << i;
    i++;
}
```

Il debugger in VS Code

VS Code ha un **debugger** integrato che ti permette di:

- Fermare il programma a una riga specifica (**breakpoint**)
- Vedere il valore di tutte le variabili in quel momento
- Eseguire il codice **una riga alla volta**

Come usarlo:

1. Clicca sul margine sinistro accanto al numero di riga per impostare un breakpoint (appare un punto rosso)
2. Premi **F5** per avviare il debug

3. Usa **F10** per eseguire riga per riga, **F11** per entrare dentro una funzione

Strategia di debugging

Quando hai un bug, segui questi passi:

1. **Riproduci il bug:** assicurati di riuscire a farlo apparire in modo affidabile
2. **Isola il problema:** dove esattamente inizia ad andare storto? Aggiungi stampe per scoprirlo
3. **Formula un'ipotesi:** cosa pensi stia causando il bug?
4. **Testa l'ipotesi:** modifica il codice e verifica se il bug sparisce
5. **Verifica che non hai rotto altro:** testa anche gli altri casi

Esempio: trovare un bug

```
// Programma con due bug: calcola la media di un array
#include <iostream>
using namespace std;

int main() {
    int voti[] = {8, 7, 9, 6, 10};
    int n = 5;
    int somma = 0;

    for (int i = 0; i <= n; i++) {    // BUG 1: <= invece di < → legge fuori
dall'array
        cout << "[DEBUG] i=" << i << ", voto=" << voti[i] << endl;
        somma += voti[i];
    }

    double media = somma / n;    // BUG 2: divisione intera → risultato
arrotondato
    cout << "Media: " << media << endl;
    return 0;
}

// Versione corretta:
// for (int i = 0; i < n; i++) {        ← usa < invece di <=
// double media = (double)somma / n;    ← converti prima di dividere
```

Con le stampe di debug, il bug 1 è visibile subito: quando `i=5`, `voti[5]` stampa un numero strano (non appartiene all'array).

Gestione degli Errori

Come gestire gli errori di base in C++ senza eccezioni avanzate.

Cosa succede quando il programma va in errore?

Tutti i programmi possono ricevere input sbagliati, trovare file mancanti o fare operazioni impossibili (come dividere per zero). Un programma che "esplode" senza spiegazioni è inutilizzabile.

La **gestione degli errori** è l'arte di intercettare queste situazioni e rispondere in modo sensato, senza far crashare il programma.

I tre tipi di errori

- **Errori di compilazione:** il codice non è scritto correttamente. Il compilatore li trova prima ancora di eseguire il programma (es. hai dimenticato un `;`).
- **Errori di runtime:** il codice è scritto bene, ma succede qualcosa di sbagliato durante l'esecuzione (es. divisione per zero, file non trovato).
- **Errori logici:** il programma gira senza crashare, ma produce risultati sbagliati. Sono i più difficili da trovare.

Gestione semplice: codici di ritorno

Un approccio comune è far restituire alle funzioni `true` se tutto è andato bene, `false` se c'è stato un errore:

```

// La funzione restituisce true se riesce, false se c'è un errore
bool dividi(int a, int b, int& risultato) {
    if (b == 0) {
        return false;    // errore: non si può dividere per zero
    }
    risultato = a / b;
    return true;
}

int main() {
    int risultato;

    if (dividi(10, 2, risultato)) {
        cout << "Risultato: " << risultato << endl;    // 5
    } else {
        cout << "Errore: divisione per zero!" << endl;
    }

    if (dividi(10, 0, risultato)) {
        cout << "Risultato: " << risultato << endl;
    } else {
        cout << "Errore: divisione per zero!" << endl;    // questo viene
stampato
    }

    return 0;
}

```

Validare l'input dell'utente

L'errore più comune è ricevere dati non validi dall'utente. Controlla sempre prima di usare:

```
int leggiEtaValida() {
    int eta;
    do {
        cout << "Inserisci la tua eta (0-120): ";
        cin >> eta;
        if (eta < 0 || eta > 120) {
            cout << "Eta non valida. Riprova." << endl;
        }
    } while (eta < 0 || eta > 120);
    return eta;
}
```

Controllare se `cin` ha letto correttamente

Se l'utente inserisce una lettera dove ti aspetti un numero, `cin` va in errore. Puoi verificarlo:

```
int numero;
cout << "Inserisci un numero: ";

if (cin >> numero) {
    cout << "Hai inserito: " << numero << endl;
} else {
    cout << "Errore: non hai inserito un numero valido." << endl;
    cin.clear(); // resetta lo stato di errore di cin
    cin.ignore(1000, '\n'); // scarta il testo non valido
}
```

Le eccezioni: per errori gravi

Le **eccezioni** sono il meccanismo del C++ per gestire errori che interrompono il flusso normale. Funzionano con tre parole chiave: `try`, `catch`, `throw`.

- `throw` lancia un errore
- `try` racchiude il codice da monitorare
- `catch` intercetta l'errore e gestisce la situazione


```

#include <stdexcept>
using namespace std;

double dividi(double a, double b) {
    if (b == 0) {
        throw runtime_error("Divisione per zero!"); // lancia l'errore
    }
    return a / b;
}

int main() {
    try {
        cout << dividi(10, 2) << endl; // 5 - va bene
        cout << dividi(10, 0) << endl; // lancia l'eccezione
    } catch (runtime_error& e) {
        // Il programma non crasha - gestiamo l'errore qui
        cout << "Errore catturato: " << e.what() << endl;
    }

    cout << "Il programma continua normalmente." << endl;
    return 0;
}

```

Tipi di eccezioni già pronti

La libreria `<stdexcept>` fornisce tipi di errore già pronti da usare:

```

#include <stdexcept>

throw runtime_error("Errore generico durante l'esecuzione");
throw invalid_argument("L'argomento passato non è valido");
throw out_of_range("Il valore è fuori dall'intervallo consentito");

```

Esempio pratico: radice quadrata sicura

```
#include <iostream>
#include <cmath>
#include <stdexcept>
using namespace std;

// Calcola la radice quadrata, ma solo per numeri non negativi
double radiceQuadrata(double x) {
    if (x < 0) {
        throw invalid_argument("Impossibile calcolare la radice di un
numero negativo");
    }
    return sqrt(x);
}

int main() {
    double valori[] = {16.0, -4.0, 25.0, 0.0};

    for (double v : valori) {
        try {
            cout << "sqrt(" << v << ") = " << radiceQuadrata(v) << endl;
        } catch (invalid_argument& e) {
            cout << "Errore per " << v << ": " << e.what() << endl;
        }
    }

    return 0;
}
```

Output:

```
sqrt(16) = 4
Errore per -4: Impossibile calcolare la radice di un numero negativo
sqrt(25) = 5
sqrt(0) = 0
```

Controllare l'apertura di un file

Prima di leggere o scrivere un file, verifica sempre che sia stato aperto:

```
#include <fstream>

ifstream file("dati.txt");

if (!file.is_open()) {
    cerr << "Errore: impossibile aprire il file 'dati.txt'" << endl;
    return 1;    // restituisce 1 per indicare che c'è stato un errore
}

// Qui sei sicuro che il file è aperto correttamente
```

`cerr` funziona come `cout`, ma manda il messaggio al canale degli errori — utile per distinguere i messaggi di errore dall'output normale.

Regole pratiche

- Valida sempre l'input dell'utente prima di usarlo
- Controlla sempre il risultato delle operazioni che possono fallire (apertura file, divisioni, ecc.)
- Non ignorare mai un errore in silenzio — il programma sembrerà funzionare ma darà risultati sbagliati
- Scrivi messaggi di errore chiari: l'utente deve capire cosa è andato storto
- Usa `cerr` per i messaggi di errore, non `cout`

Leggere e Scrivere File

Come leggere e scrivere file in C++ con `fstream`.

Leggere e scrivere file in C++

Quando il tuo programma termina, tutte le variabili spariscono. Se vuoi **conservare i dati** — i voti degli studenti, un punteggio di un gioco, una lista della spesa — devi salvarli su un file.

Con i file puoi anche leggere dati preparati in anticipo, senza dover inserire tutto a mano ogni volta.

Le tre classi per i file

In C++ si usa la libreria `<fstream>` (abbreviazione di "file stream"):

Classe	Per cosa serve
<code>ofstream</code>	Scrivere su file
<code>ifstream</code>	Leggere da un file
<code>fstream</code>	Leggere E scrivere

Scrivere su un file

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream file("saluto.txt");    // apre (o crea) il file in scrittura

    if (file.is_open()) {          // controlla sempre che sia aperto!
        file << "Ciao, mondo!" << endl;
        file << "Questa è la seconda riga." << endl;
        file.close();              // chiudi il file quando hai finito
        cout << "File scritto con successo." << endl;
    } else {
        cout << "Errore: impossibile aprire il file." << endl;
    }

    return 0;
}
```

Questo crea il file `saluto.txt` nella stessa cartella del programma. Se esiste già, viene **sovrascritto**.

Aggiungere testo senza cancellare

Per aggiungere righe in fondo a un file già esistente (senza cancellarlo), usa `ios::app` :

```
ofstream file("diario.txt", ios::app); // "app" = append (aggiungi in coda)

if (file.is_open()) {
    file << "Oggi ho imparato i file in C++." << endl;
    file.close();
}
```

Leggere da un file

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    ifstream file("saluto.txt"); // apre il file in lettura

    if (file.is_open()) {
        string riga;
        while (getline(file, riga)) { // legge una riga alla volta
            cout << riga << endl;
        }
        file.close();
    } else {
        cout << "Errore: file non trovato." << endl;
    }

    return 0;
}
```

`getline(file, riga)` legge una riga intera. Quando non ci sono più righe, il ciclo si ferma da solo.

Leggere parola per parola o numero per numero

Se il file contiene dati separati da spazi o da righe, puoi leggere con `>>` :

```

// Leggere parole
ifstream file("parole.txt");
string parola;
while (file >> parola) {    // legge una parola alla volta
    cout << parola << endl;
}
file.close();

// Leggere numeri e calcolare la media
ifstream file2("numeri.txt");
int numero, somma = 0, contatore = 0;
while (file2 >> numero) {
    somma += numero;
    contatore++;
}
if (contatore > 0) {
    cout << "Media: " << (double)somma / contatore << endl;
}
file2.close();

```

Leggere più valori per riga

Se ogni riga del file contiene più valori (es. nome e voto separati da spazio):

```

// Il file "dati.txt" contiene:
// Alice 8.5
// Bob 7.2
// Carlo 9.0

ifstream file("dati.txt");
string nome;
double voto;

while (file >> nome >> voto) {    // legge nome e voto insieme
    cout << nome << ": " << voto << endl;
}
file.close();

```

Esempio pratico: registro voti su file

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

// Chiede i voti all'utente e li salva nel file
void salvaVoti(const string& nomefile) {
    ofstream file(nomefile);
    if (!file.is_open()) {
        cout << "Errore nella creazione del file." << endl;
        return;
    }

    int n;
    cout << "Quanti studenti? ";
    cin >> n;

    for (int i = 0; i < n; i++) {
        string nome;
        double voto;
        cout << "Nome studente " << (i+1) << ": ";
        cin >> nome;
        cout << "Voto: ";
        cin >> voto;

        file << nome << " " << voto << endl;    // scrive una riga per
    studente
    }

    file.close();
    cout << "Dati salvati in " << nomefile << endl;
}

// Legge il file e stampa i voti con la media
void leggiVoti(const string& nomefile) {
    ifstream file(nomefile);
    if (!file.is_open()) {
        cout << "File non trovato: " << nomefile << endl;
        return;
    }

    cout << "\n=== Voti salvati ===" << endl;
    string nome;
    double voto, somma = 0;
```

```

int contatore = 0;

while (file >> nome >> voto) {
    cout << nome << ": " << voto << endl;
    somma += voto;
    contatore++;
}

if (contatore > 0) {
    cout << "Media classe: " << somma / contatore << endl;
}

file.close();
}

int main() {
    string nomefile = "registro.txt";
    salvaVoti(nomefile);    // salva i voti su file
    leggiVoti(nomefile);    // rilegge e stampa
    return 0;
}

```

Regole pratiche

- Verifica **sempre** `file.is_open()` prima di usare il file
- Chiudi il file con `file.close()` quando hai finito
- Se dimentichi di chiudere, il file viene chiuso automaticamente quando l'oggetto viene distrutto (ma è buona abitudine chiuderlo esplicitamente)
- Per aggiungere contenuto senza cancellare, usa `ios::app`
- Usa `cerr` per i messaggi di errore relativi ai file, non `cout`

Librerie Standard

Le librerie standard del C++ più utili per iniziare a programmare.

Non devi inventare tutto da zero

Non devi inventare tutto da zero. Il C++ include una collezione enorme di strumenti già pronti — funzioni matematiche, gestione dei file, algoritmi di ordinamento e molto altro.

Si chiama **libreria standard** (Standard Library) ed è disponibile su ogni computer con un compilatore C++. Basta includerla con `#include` .

<iostream> — Input e Output

La libreria indispensabile: serve per stampare a schermo e leggere da tastiera.

```
#include <iostream>
using namespace std;

int main() {
    cout << "Ciao!" << endl;    // stampa a schermo

    int x;
    cin >> x;                    // legge un valore da tastiera

    cerr << "Errore!" << endl;  // stampa un messaggio di errore

    return 0;
}
```

- `cout` — stampa a schermo
- `cin` — legge da tastiera
- `cerr` — stampa messaggi di errore (canale separato)
- `endl` — va a capo

<string> — Testo

Per usare il tipo `string` e tutte le sue funzionalità:

```
#include <string>
using namespace std;

string nome = "Alice";
int lunghezza = nome.length();    // 5
string inizio = nome.substr(0, 3); // "Ali"

// Converta numero in stringa e viceversa
string s = to_string(42);    // "42"
int n = stoi("123");        // 123
double d = stod("3.14");    // 3.14
```

<cmath> — Funzioni Matematiche

Contiene funzioni matematiche avanzate (potenza, radice, arrotondamento, trigonometria):

```
#include <cmath>
using namespace std;

// Potenza: 2 elevato alla 10
cout << pow(2, 10) << endl;    // 1024

// Radice quadrata
cout << sqrt(25) << endl;      // 5
cout << sqrt(2) << endl;      // 1.41421

// Valore assoluto
cout << abs(-5) << endl;      // 5
cout << fabs(-3.14) << endl;  // 3.14 (per numeri decimali)

// Arrotondamento
cout << floor(3.7) << endl;    // 3 (arrotonda in giù)
cout << ceil(3.2) << endl;    // 4 (arrotonda in su)
cout << round(3.5) << endl;    // 4 (arrotonda al più vicino)

// Costante PI greco
cout << M_PI << endl;        // 3.14159265...
```

<cstdlib> e <ctime> — Numeri Casuali e Utilità

```
#include <cstdlib>
#include <ctime>
using namespace std;

// Inizializza il generatore di numeri casuali (fallo una sola volta)
srand(time(0));

int casuale = rand() % 100; // numero casuale tra 0 e 99
int dado = rand() % 6 + 1; // numero casuale tra 1 e 6

// Conversione da stringa a numero (alternativa a stoi/stod)
int n = atoi("42"); // stringa → intero
double d = atof("3.14"); // stringa → decimale
```

<cctype> — Funzioni sui Caratteri

Per controllare e trasformare singoli caratteri:

```
#include <cctype>
using namespace std;

cout << isupper('A') << endl; // 1 (vero): è maiuscolo?
cout << islower('a') << endl; // 1 (vero): è minuscolo?
cout << isdigit('5') << endl; // 1 (vero): è una cifra?
cout << isalpha('x') << endl; // 1 (vero): è una lettera?
cout << isspace(' ') << endl; // 1 (vero): è uno spazio?

cout << (char)tolower('A') << endl; // 'a'
cout << (char)toupper('a') << endl; // 'A'
```

<algorithm> — Algoritmi Pronti

Contiene algoritmi già implementati per lavorare con collezioni di dati:

```

#include <algorithm>
#include <vector>
using namespace std;

vector<int> v = {5, 2, 8, 1, 9, 3};

// Ordina dal più piccolo al più grande
sort(v.begin(), v.end());
// v è ora: {1, 2, 3, 5, 8, 9}

// Ordina dal più grande al più piccolo
sort(v.begin(), v.end(), greater<int>());
// v è ora: {9, 8, 5, 3, 2, 1}

// Trova il minimo e il massimo
cout << *min_element(v.begin(), v.end()) << endl; // 1
cout << *max_element(v.begin(), v.end()) << endl; // 9

// Cerca un elemento
auto it = find(v.begin(), v.end(), 5);
if (it != v.end()) {
    cout << "Trovato!" << endl;
}

// Inverte l'ordine degli elementi
reverse(v.begin(), v.end());

```

<vector> — Array Dinamici

Per usare array che cambiano dimensione (vedi la sezione dedicata):

```

#include <vector>
using namespace std;

vector<int> v = {1, 2, 3};
v.push_back(4); // aggiunge 4 alla fine
v.push_back(5);
cout << v.size() << endl; // 5

```

<limits> — Valori Massimi e Minimi dei Tipi

Per sapere i limiti di ogni tipo numerico:

```
#include <limits>
using namespace std;

cout << numeric_limits<int>::max() << endl;    // 2147483647 (il più grande
int)
cout << numeric_limits<int>::min() << endl;    // -2147483648 (il più
piccolo int)
cout << numeric_limits<double>::max() << endl; // ~1.8×10^308
```

Tabella riassuntiva

Libreria	Cosa contiene
<iostream>	cout, cin, cerr, endl
<string>	string, getline, stoi, stod, to_string
<cmath>	sqrt, pow, abs, floor, ceil, M_PI
<algorithm>	sort, find, min_element, max_element, reverse
<vector>	vector
<fstream>	ofstream, ifstream, fstream
<cstdlib>	rand, srand, atoi, atof
<cctype>	isupper, islower, isdigit, tolower, toupper
<limits>	numeric_limits
<ctime>	time